

DOCUMENTATION TECHNIQUE FANTASYREALM ONLINE

Système de gestion de personnages — PixelVerse Studios

1. Réflexions technologiques initiales
2. Configuration de l'environnement de travail
3. Modèle conceptuel de données & diagramme de classe
4. Diagramme d'utilisation (cas d'usage)
5. Diagramme de séquence
6. Documentation du déploiement

ECF · TITRE PROFESSIONNEL DÉVELOPPEUR WEB ET WEB MOBILE ·
JUILLET 2026

1. RÉFLEXIONS TECHNOLOGIQUES INITIALES

1.1 Front-end

HTML / CSS / JavaScript natifs, sans framework. Ce choix se justifie par la taille du projet (une dizaine de pages, un seul développeur) : un framework (React, Vue) aurait ajouté une couche de complexité (build, transpilation) sans bénéfice proportionné, alors que le référentiel du titre professionnel évalue justement la capacité à structurer une interface dynamique en JavaScript pur. Les données sont d'abord simulées en `localStorage` pour livrer un prototype testable de bout en bout indépendamment du back-end, avant intégration à la véritable API.

1.2 Back-end

PHP 8.1 + PDO, sans framework, en architecture simple Contrôleur / Repository. PHP reste le langage le plus largement supporté par les hébergeurs mutualisés bon marché, ce qui facilite le déploiement final exigé par le client. PDO avec requêtes préparées (`ATTR_EMULATE_PREPARES => false`) protège nativement contre les injections SQL sans dépendance externe.

1.3 Bases de données : pourquoi une base relationnelle ET une base non relationnelle

Donnée	Base choisie	Justification
Utilisateurs, personnages, articles, commentaires	MySQL / MariaDB	Données fortement relationnelles (un personnage appartient à un utilisateur, dispose de plusieurs articles, etc.). Les contraintes de clé étrangère (ON DELETE CASCADE) garantissent l'intégrité référentielle exigée par les règles de gestion (suppression en cascade d'un compte joueur, par exemple).
Logs d'activité (suivi admin)	MongoDB	Écriture en append-only, jamais modifiée ni jointe à d'autres tables, avec une structure pouvant varier selon le type d'action journalisée. Un schéma flexible (document JSON) convient mieux qu'une table SQL figée, et le volume potentiellement important de logs n'a pas besoin de transactions ACID complexes.

1.4 Sécurité

- Mots de passe hashés avec `password_hash()` (bcrypt), jamais stockés en clair
- Règle CNIL appliquée à la création de compte (8 caractères min., majuscule, minuscule, chiffre, caractère spécial)
- 100% des requêtes SQL passent par des requêtes préparées PDO
- Autorisations vérifiées côté serveur à chaque route sensible (jamais uniquement côté client)
- Sessions PHP avec régénération de l'identifiant à la connexion (anti-fixation de session)
- CORS restreint à une origine explicite (pas de wildcard) pour l'usage de cookies de session

1.5 Écart assumé par rapport au MCD fourni

Le champ password est déclaré en VARCHAR(255) plutôt qu'en VARCHAR(50) comme indiqué dans le MCD initial, afin d'accueillir un hash bcrypt (60 caractères). De même, un champ `date_creation` a été ajouté à la table personnage (absent du MCD fourni) car indispensable au filtre « plage de date de création » exigé par l'énoncé. Ces deux écarts sont les seuls par rapport au schéma fourni en annexe.

2. CONFIGURATION DE L'ENVIRONNEMENT DE TRAVAIL

2.1 Outils utilisés

Outil	Rôle
PHP 8.1+ (extensions pdo_mysql, mongodb)	Exécution du back-end
Composer	Gestion de la dépendance <code>mongodb/mongodb</code>
MySQL / MariaDB	Base de données relationnelle
MongoDB (local ou Atlas)	Base de données non relationnelle (logs)
Git / GitHub	Gestion de version, dépôt public du code
Éditeur de code (VS Code)	Développement
Navigateur (Chrome/Firefox) + outils développeur	Tests front-end, accessibilité, réseau

2.2 Installation en local (résumé — détails complets dans le README du back-end)

```
composer install
cp .env.example .env # renseigner les identifiants MySQL / MongoDB
mysql -u root -p -e "CREATE DATABASE fantasyrealm CHARACTER SET utf8mb4;"
mysql -u root -p fantasyrealm < sql/schema.sql
mysql -u root -p fantasyrealm < sql/seed.sql
php -S localhost:8000 -t public # serveur de developpement
```

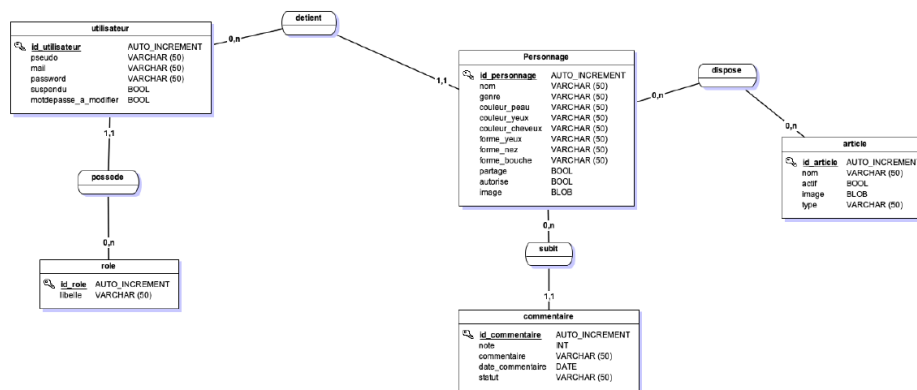
2.3 Variables d'environnement (.env)

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=fantasyrealm
DB_USER=root
DB_PASSWORD=
MONGO_URI=mongodb://127.0.0.1:27017
MONGO_DB=fantasyrealm_logs
FRONTEND_ORIGIN=http://localhost:5500
```

3. MODÈLE CONCEPTUEL DE DONNÉES & DIAGRAMME DE CLASSE

3.1 MCD fourni (Annexe 1 de l'énoncé)

Annexe 1 : schémas de base de données (MCD)



Modèle Conceptuel de Données fourni en annexe de l'ECF

3.2 Diagramme de classe (vue objet du domaine)

Traduction orientée objet du MCD, telle qu'implémentée côté back-end (classes Repository / entités manipulées par l'API). Les cardinalités des associations reprennent celles du MCD fourni.

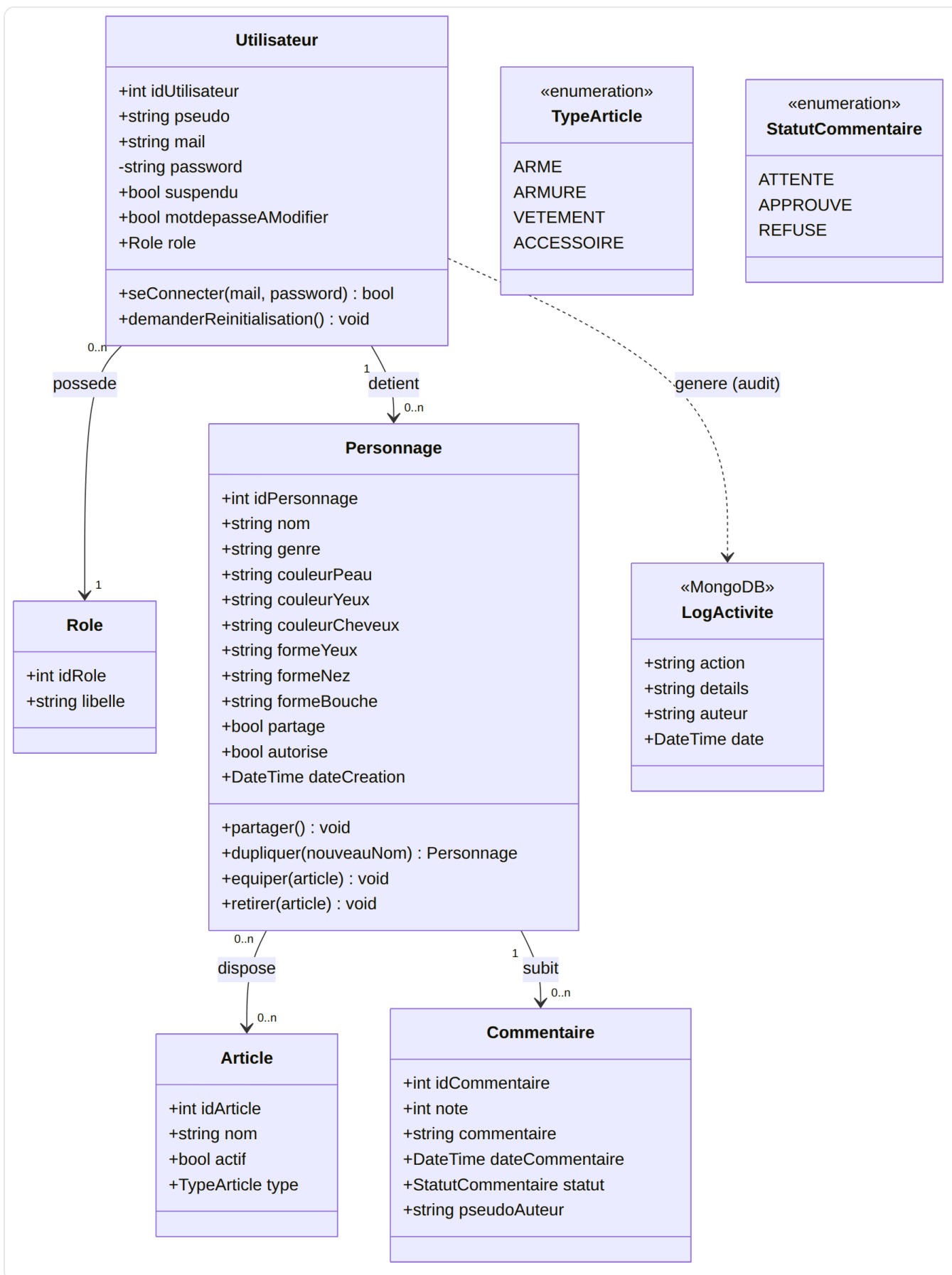


Diagramme de classe du domaine FantasyRealm Online

4. DIAGRAMME D'UTILISATION (CAS D'USAGE)

Quatre acteurs : le **Visiteur** non connecté, le **Joueur**, l' **Employé** et l'**Administrateur** — ce dernier héritant de toutes les actions de l'employé (généralisation UML), auxquelles s'ajoutent la gestion des comptes employés et la consultation des logs.

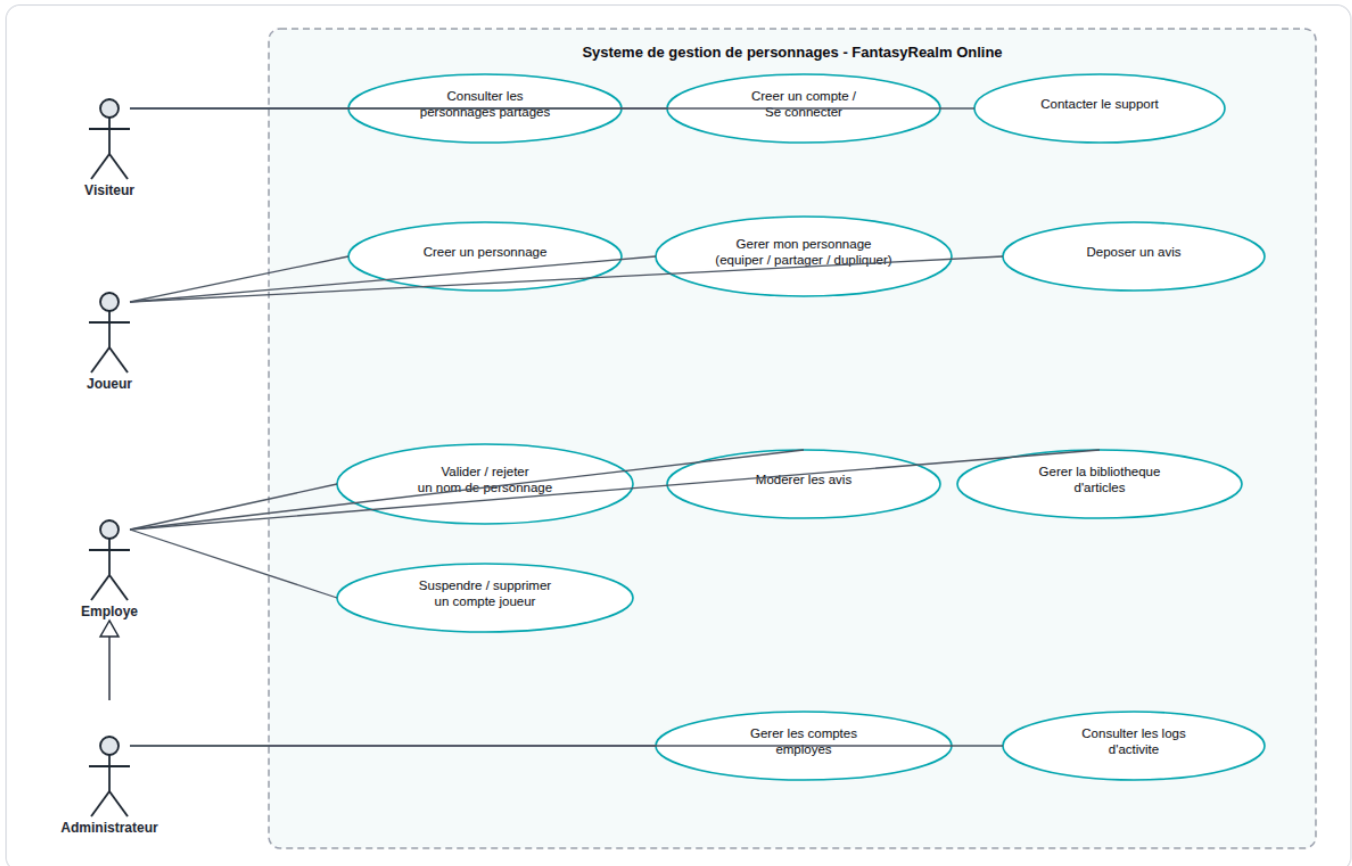
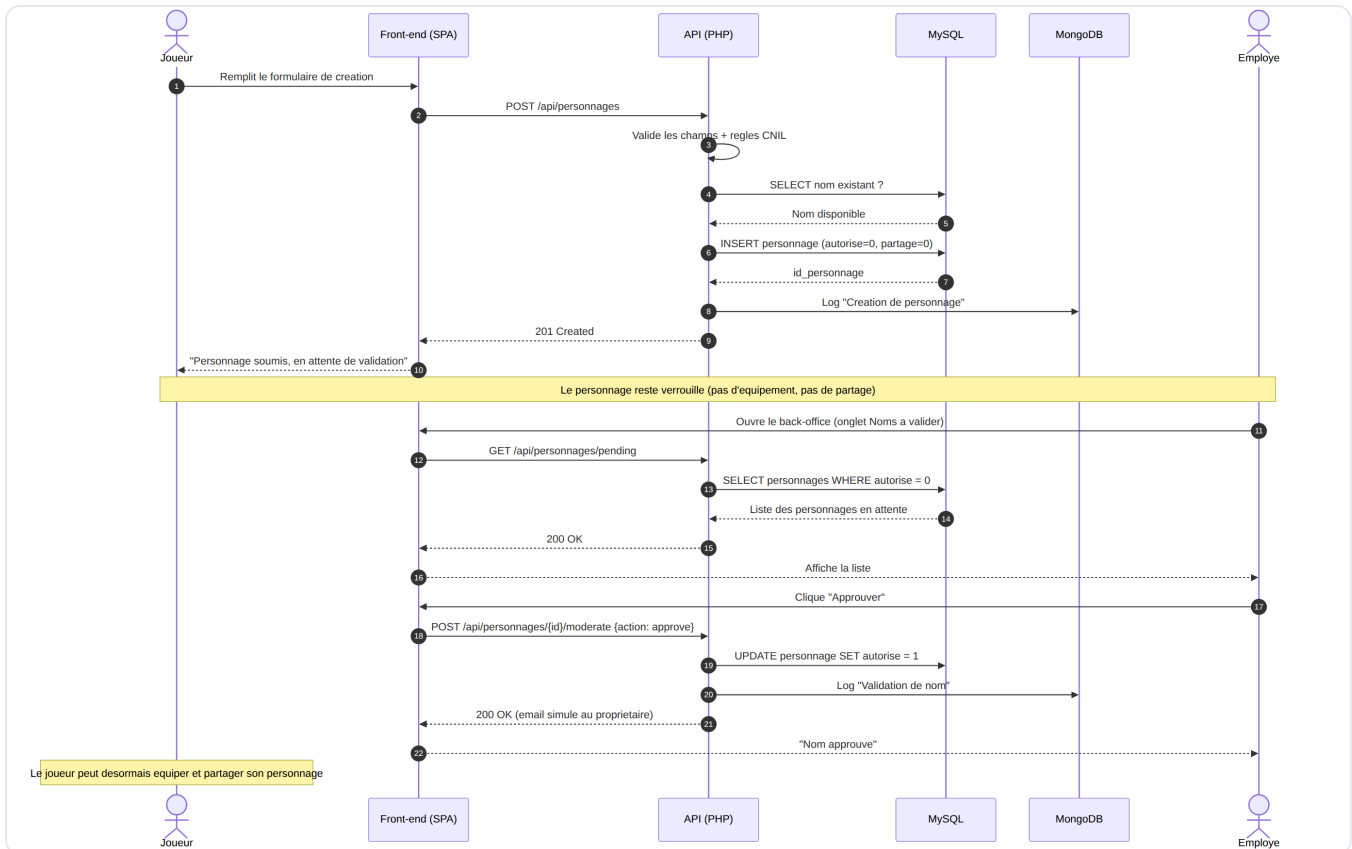


Diagramme d'utilisation — FantasyRealm Online

5. DIAGRAMME DE SÉQUENCE

Flux complet de **création puis validation d'un personnage**, illustrant l'interaction entre le front-end, l'API, MySQL (persistance métier) et MongoDB (journalisation).



Séquence : création et validation du nom d'un personnage

6. DOCUMENTATION DU DÉPLOIEMENT

6.1 Cibles de déploiement retenues

L'énoncé suggère fly.io, Heroku, Azure ou Vercel. À la vérification (juillet 2026), fly.io a supprimé son offre gratuite fin 2024 et Heroku la sienne dès 2022 : ces pistes ont donc été écartées au profit de services offrant une offre gratuite réellement active et vérifiée à la date du déploiement, sans carte bancaire requise.

Composant	Service	Offre gratuite
Front-end statique	Netlify	Illimité, sans CB
Back-end PHP (Docker)	Render (Web Service)	750h/mois, sans CB (mise en veille après 15 min d'inactivité)
Base MySQL	Aiven	Gratuite à vie, sans CB (1 Go)
Base MongoDB	MongoDB Atlas (cluster M0)	Gratuite à vie, sans CB (512 Mo)

Pourquoi Docker pour le back-end : l'extension PHP `mongodb` n'est pas proposée par les hébergeurs mutualisés classiques. Un déploiement par conteneur (`Dockerfile` fourni à la racine de `backend/`) garantit sa disponibilité en installant l'extension via PECL au moment du build.

6.2 Étapes de déploiement

1. Créer la base MySQL sur Aiven, récupérer les identifiants de connexion, importer `sql/schema.sql` puis `sql/seed.sql` (via un client MySQL ou phpMyAdmin en ligne)
2. Créer un cluster gratuit (M0) sur MongoDB Atlas, créer un utilisateur applicatif, autoriser l'accès réseau (0.0.0.0/0 pour un contexte de démonstration) et récupérer l'URI de connexion
3. Sur Render : créer un nouveau "Web Service", le connecter au dépôt GitHub, dossier racine `backend/` , environnement "Docker" (le `Dockerfile` fourni est détecté automatiquement)
4. Renseigner les variables d'environnement sur Render (onglet Environment) : `DB_HOST` , `DB_PORT` , `DB_NAME` , `DB_USER` , `DB_PASSWORD` (fournis par Aiven), `MONGO_URI` , `MONGO_DB` (fournis par Atlas), `FRONTEND_ORIGIN`
5. Déployer : Render construit l'image Docker et expose une URL (voir l'URL réelle en section 6.4)
6. Sur Netlify : "Add new site" → "Import from GitHub", dossier de publication `frontend/` , aucune commande de build nécessaire (site statique)
7. Mettre à jour `FRONTEND_ORIGIN` sur Render avec l'URL réelle générée par Netlify
8. Vérifier les parcours critiques avec les comptes de démonstration (voir manuel d'utilisation)

6.3 Points de vigilance en production

- Le service Render gratuit se met en veille après 15 minutes d'inactivité : la première requête après une veille peut prendre jusqu'à une minute (cold start) — à mentionner au jury le jour de la présentation
- Forcer HTTPS (cookies de session en `Secure`) — Render et Netlify fournissent un certificat TLS automatique
- Ne jamais exposer publiquement `.env`, `composer.json` ou le dossier `sql/`
- Limiter les IP autorisées sur Aiven et MongoDB Atlas si possible (sinon accepter 0.0.0.0/0 pour la durée de l'évaluation uniquement)

6.4 Déploiement effectif — URLs de production

Composant	Service	URL
Front-end	Netlify	<code>https://fantasyrealmonline.netlify.app</code>
Back-end (API)	Render	<code>https://fantasyrealmonline.onrender.com</code>
Code source	GitHub (public)	<code>https://github.com/Herumoru/FantasyRealmOnline</code>

Retour d'expérience du déploiement : trois difficultés imprévues ont été rencontrées et documentées ici pour référence :

- La compilation de l'extension PECL `mongodb` nécessite les paquets de compilation (`$PHPIZE_DEPS` : `gcc`, `make`, `autoconf...`), absents de l'image `php:8.1-apache` de base.
- PECL ne distribue plus que la branche 2.x de l'extension `mongodb` ; la librairie applicative `mongodb/mongodb` doit donc être contrainte en `^2.3` (et non `^1.17`) pour rester compatible, ces deux versions évoluant de façon synchronisée.
- Un handshake TLS échouait systématiquement vers MongoDB Atlas : la cause n'était ni Docker ni OpenSSL, mais simplement une **liste d'accès réseau (Network Access) vide** côté Atlas — aucune IP n'y était encore autorisée.